MODULE 19 · DOMAIN 2 - PROCESS

Scope in adaptive projects

The product backlog, user stories, acceptance criteria, definitions of ready and done, splitting, story mapping and ordering.

DOMAIN II - Process EXAM WEIGHT 41% MODULE 19 of 49

admin@sthithapragna.com

In adaptive work, scope is **discovered**, not frozen. The backlog is how it is managed.

How scope behaves: predictive or adaptive

PREDICTIVE

Scope fixed, estimate the rest

- Scope defined and baselined early
- Time and cost estimated from scope
- Change through change control
- WBS and scope statement

ADAPTIVE

Time and cost fixed, scope flexes

- Scope discovered and refined
- Timeboxes and team size often fixed
- Change through the backlog
- Backlog, stories and a product goal

THE TIEBREAKER

Adaptive projects often fix time and cost and let scope flex. The highest-value scope is delivered first.

From big ideas to small work

01
Theme
A broad goal: easier appointment booking.

02
Epic
A large body of work: online rescheduling.

03
Feature
A usable capability: reschedule from a reminder.

04
User story
Fits in an iteration: one small slice of value.

05
Task
The team's own breakdown of how to build it.

ON THE EXAM

Only stories near the top of the backlog need full detail. Epics further down can stay large.

User stories

- **As a** patient, **I want** to reschedule from my reminder message, **so that** I do not have to phone the clinic.
- Who wants it, what they want, and **why**. The 'so that' is the value.
- **Card, conversation, confirmation**: a short note, a discussion, and agreed acceptance criteria.
- A story is a placeholder for a conversation, not a complete specification.

> ON THE EXAM
>
> The conversation matters more than the card. Stories are refined by talking, not by writing more.

INVEST: what makes a good story

LETTER	MEANS	TEST
I	Independent	Can it be built and released without waiting on another story?
N	Negotiable	Is the detail still open to discussion?
V	Valuable	Does it deliver value to a user or customer?
E	Estimable	Can the team size it?
S	Small	Can it be finished within one iteration?
T	Testable	Can we tell when it is done?

ON THE EXAM

A story that fails 'small' or 'estimable' needs splitting or a spike, not a longer iteration.

Acceptance criteria

- The conditions a story must meet for the product owner to accept it.
- Often written as **Given, When, Then**:
- **Given** a booked appointment more than 24 hours away, **when** I choose a new slot from my reminder, **then** the old slot is released and I get a confirmation.
- Agreed **before** work starts, so the team knows what done looks like for this item.

> ON THE EXAM
>
> Acceptance criteria agreed after the work is built are just a rejection waiting to happen.

Definition of ready or definition of done?

DEFINITION OF READY

Before work starts

- Story is small enough
- Acceptance criteria agreed
- Dependencies known
- Team understands it well enough to plan

DEFINITION OF DONE

Before work counts as finished

- Meets acceptance criteria
- Tested, reviewed, integrated
- Documentation updated
- Meets quality and non-functional standards

THE TIEBREAKER

Ready controls what enters an iteration. Done controls what leaves it. Both are agreed by the team.

Backlog refinement

- Continuous: items are clarified, split, estimated and reordered, **ahead** of planning.
- Done by the product owner and the developers together.
- A useful backlog is **DEEP**: detailed appropriately, estimated, emergent, prioritised.
- Refinement squeezed into sprint planning turns planning into a marathon.

> ON THE EXAM
>
> Long, painful planning sessions are usually a refinement problem. Refine ahead, continuously.

Splitting stories: vertical slices

SPLIT BY	EXAMPLE
Workflow step	Search for a slot; then choose it; then confirm it
Business rule	Standard appointments first; urgent ones later
Data variation	One clinic first; then all clinics
Happy path first	Successful booking first; error handling next
Interface	Web first; kiosk later

ON THE EXAM

Split vertically, so each slice works end to end. Splitting by layer, such as database then screen, delivers nothing usable.

Story mapping

- Arrange user activities left to right as the **backbone** of the user's journey.
- Stack stories under each activity, most important at the top.
- Draw horizontal lines to slice **releases**: the top slice across the whole journey is the first release.
- Shows stakeholders the whole product, and what arrives when, far better than a long flat list.

> ON THE EXAM
>
> A long flat backlog hides the journey. A story map shows it, and makes the first release obvious.

Ordering the backlog

- **Value**: what matters most to customers and the business.
- **Risk**: tackle the riskiest, most uncertain items early, while there is time to react.
- **Dependencies** and **learning**: what unlocks other work, or answers big questions.
- **Cost of delay**: what loses value if it waits.
- The **product owner** decides the order, informed by stakeholders and the team.

> ON THE EXAM
>
> Order by value, risk and dependency, not by who shouted loudest. Risky items early, not last.

Where non-functional requirements go

- Standards that apply to everything, such as security and accessibility, go in the **definition of done**.
- Standards for one feature go in that story's **acceptance criteria**.
- Larger pieces of work, such as a performance improvement, can be **backlog items** of their own.
- If no story or standard mentions performance, nobody will build for it.

> ON THE EXAM
>
> Non-functional requirements must live somewhere visible. Otherwise they surface only after release.

Changing scope in adaptive work

- New ideas become **backlog items**; the product owner orders them against everything else.
- Within fixed time and cost, adding something usually means something lower moves down or out.
- Formal change requests still apply to **contracts, budgets and governance baselines**.
- Changing the current iteration's work is limited: only if the iteration goal is safe.

> ON THE EXAM
>
> The backlog is the change mechanism for product scope. Contracts and funding still need formal change.

Scope on hybrid projects

- A **WBS** for the predictive work, and a **backlog** for the adaptive work.
- The WBS can hold the adaptive work as a **release or epic** with its milestones.
- Governance baselines are set at the level of releases and outcomes, not individual stories.
- Integration points, such as a hardware-ready date, are dependencies in the backlog order.

> ON THE EXAM
>
> Baseline the release, not every story. Let the backlog flex inside the release boundary.

Project Lantern: the app backlog

- **Story map backbone**: find a clinic, book, get reminded, reschedule, check in at the kiosk.
- **First release slice**: book and cancel standard appointments at one clinic, on web.
- **Definition of done** includes accessibility checks and page loads under two seconds.
- The riskiest item, the booking-system integration, was built in the second sprint, not the last.

> ON THE EXAM
>
> A journey-shaped backlog, a thin first slice, standards in the definition of done, and risk tackled early.

Scenario signals and the move

THE SCENARIO SAYS	THE MOVE
A story is too big for one iteration	Split it into vertical slices
Work is rejected at review as not what was wanted	Agree acceptance criteria before work starts
Sprint planning takes hours	Refine continuously, ahead of planning
A quality problem appears after release	Put the standard in the definition of done
Risky work is left until the end	Order by value, risk and dependency
Stakeholders cannot see the whole product	Facilitate a story map; slice releases

ON THE EXAM

Small, valuable, testable stories, refined ahead, ordered by value and risk, and finished to a clear standard.

> A backlog item reads 'Build the management reporting system'. The team estimates it would take at least three sprints.
>
> The product owner wants it in the next sprint.

What should the team do?

A Extend the next sprint so the item can be completed in one go.

B Split it into smaller vertical slices, such as one report at a time, each usable on its own.

C Split it by layer: the database first, then the services, then the screens.

D Start it in the next sprint and carry it over for as long as it takes.

◆ PAUSE AND DECIDE

What should the team do?

A Extend the sprint.

Sprints are fixed length. Oversized items are split, not accommodated.

B Vertical slices, one report at a time.

Each slice is small, valuable and testable, and the most important report can come first.

C Split by layer.

Horizontal slices. Nothing is usable until all three are done.

D Carry it over.

Nothing done for three sprints, and no feedback until the end.

THE SIGNAL

An epic dressed as a story: split it vertically, so value arrives every sprint.

> In the last two sprint reviews, the product owner rejected several stories that the developers considered finished. The developers say they built what the stories described.
>
> The stories have short descriptions and no acceptance criteria.

What should the project leader do?

A Ask the product owner to accept the stories, since they match the descriptions.

B Add more testing before each review.

C Write the acceptance criteria for the team personally.

D Facilitate the product owner and developers agreeing acceptance criteria before work starts on each story.

◆ PAUSE AND DECIDE

What should the project leader do?

A Pressure the PO to accept.

The PO owns acceptance. The gap in understanding remains.

B More testing.

Tests against what? Without criteria, nobody knows what to test for.

C PM writes the criteria.

The PM does not own the product's needs. The PO and team must agree them.

D Agree acceptance criteria before work.

Confirmation in the three Cs: a shared definition of done for each story.

THE SIGNAL

Rejection at review with no acceptance criteria: the confirmation was never agreed. Agree it first.

> Sprint planning for a two-week sprint now takes most of a day. The team spends hours clarifying stories, splitting them and estimating for the first time.
>
> The developers say they dread planning day.

What should the project leader do?

A Help the team hold regular refinement sessions ahead of planning, with a definition of ready.

B Extend the planning timebox, since the work needs doing.

C Skip estimation to save time in planning.

D Refine the stories personally before each planning session.

◆ PAUSE AND DECIDE

What should the project leader do?

A Refine ahead, with a definition of ready.

Moves the clarifying and splitting out of planning, continuously.

B Longer planning.

Accepts the symptom. Planning should confirm, not discover.

C Skip estimation.

Removes the information the team needs to forecast its sprint.

D PM refines alone.

Refinement needs the product owner's knowledge and the developers' sizing.

THE SIGNAL

Painful planning is a refinement problem. Refine continuously, ahead of time.

> After the first public release of a ticket-booking app, customers complain that pages take several seconds to load at busy times.
>
> No story, acceptance criterion or team standard mentioned performance.

What should the team do to prevent this in future?

A. Plan a dedicated performance-testing sprint before each major release.

B. Add performance standards to the definition of done, and to acceptance criteria where specific.

C. Buy more powerful servers for the next release.

D. Ask the product owner to take responsibility for performance.

◆ PAUSE AND DECIDE

What should the team do to prevent this in future?

A. A performance sprint before releases.
Batches quality at the end, the opposite of building it in each sprint.

B. Performance in the DoD and criteria.
Makes the non-functional requirement visible and checked on every item.

C. Bigger servers.
Treats a symptom, and may not help if the design is the cause.

D. Make it the PO's job.
Quality is the developers' accountability, through the definition of done.

THE SIGNAL

A quality attribute nobody wrote down: put it in the definition of done, so every item meets it.

> A product owner orders the backlog mainly by the seniority of whoever requested each item. The integration with a payment provider, the most technically uncertain item, sits near the bottom.
>
> The release date is fixed, four months away.

What should the project leader do?

A. Reorder the backlog personally to bring the integration forward.

B. Leave the order alone, since backlog ordering belongs to the product owner.

C. Coach the product owner on ordering by value, risk and dependencies, including bringing risky items forward.

D. Escalate the product owner's ordering to the sponsor.

◆ PAUSE AND DECIDE

What should the project leader do?

A Reorder it personally.

Takes the product owner's decision away. Wrong person.

B Leave it alone.

Right about ownership, wrong to stay silent about a serious risk.

C Coach the PO: value, risk, dependency.

Respects ownership and fixes the ordering logic that creates the risk.

D Escalate.

Too early. A coaching conversation has not happened.

THE SIGNAL

The PO owns the order; the leader coaches how to order. Risky, uncertain items belong early.

> A product's backlog has grown to three hundred items in one long list. Stakeholders say they cannot tell what the first release will contain or whether it will be usable.
>
> The product owner is struggling to explain the plan.

What should the project leader do?

A Sort the backlog by feature name so it is easier to read.

B Replace the backlog with a work breakdown structure.

C Produce a detailed Gantt chart for all three hundred items.

D Facilitate a user story map with the product owner and stakeholders, and slice the first release.

◆ PAUSE AND DECIDE

What should the project leader do?

A Sort alphabetically.

Still a flat list. The journey and the release remain invisible.

B Switch to a WBS.

Abandons adaptive scope instead of making it visible.

C A Gantt chart.

False precision for adaptive work, and it still hides the user journey.

D Story map; slice the first release.

Shows the whole journey and makes a usable first release visible.

THE SIGNAL

A long flat backlog nobody understands: map the user journey and slice releases across it.

Small, valuable, testable stories. Refined ahead. **Ordered by value and risk.**